WATERVISION

Détection et estimation des surfaces d'eau

Document de conception

À quoi sert ce document

Ce document décrit comment le logiciel WaterVision est construit. Il suit le chemin d'une image, depuis le fichier ouvert par l'utilisateur jusqu'au masque, au pourcentage d'eau et au rapport exporté. Chaque étape indique le fichier Python qui la réalise et l'idée mathématique qui la justifie.

Le logiciel est un projet de traitement numérique des images. Les opérations sont écrites avec NumPy et SciPy. L'interface est une fenêtre de bureau CustomTkinter. Pillow lit et écrit les images. Matplotlib trace les graphiques et les rapports. OpenCV n'est pas utilisé, et aucun modèle entraîné ne prend de décision.

Comment lire la suite

- Les sections 1 à 4 présentent l'objectif, les dossiers, l'état partagé et le chemin des données.
- Les sections 5 à 13 détaillent les algorithmes, dans l'ordre du calcul.
- Les sections 14 à 18 décrivent l'interface, les fils d'exécution, la démonstration et la manière de lancer le logiciel.

Pour lancer l'application : python main.py. Pour relire ce document après une modification du générateur : python scripts/generer_documentation.py.

1. Objectif

WaterVision détecte des étendues d'eau naturelles dans une photographie : lac, rivière, étang ou retenue. Il produit un masque binaire, compte les pixels d'eau, et n'annonce une surface en mètres carrés que si l'utilisateur fournit la taille réelle d'un pixel.
Par défaut, seuls les pixels bleus sont retenus, entre 185° et 255°. Le vert des continents et le jaune du sable tombent en dehors de cet arc. Le logiciel sépare la teinte de la luminosité, et il refuse de conclure à la présence d'eau à partir de la seule texture : une plage lisse a un spectre aussi pauvre en hautes fréquences qu'une eau calme.

2. Organisation du code

Le programme démarre par main.py, qui appelle watervision.app.main. Le paquet est séparé en trois responsabilités.

- watervision/processing contient les calculs. Aucun widget n'y est importé. Ces fonctions peuvent être testées sans ouvrir de fenêtre.
- watervision/state.py garde les images et les réglages d'une session. C'est le seul endroit où les grands tableaux NumPy sont conservés.
- watervision/ui et watervision/app.py construisent la fenêtre. Les pages lisent l'état et demandent à l'application de lancer un calcul.

Fichiers de calcul

- color.py : luminance BT.601 et conversion RGB vers HSV, écrites en NumPy.
- filters.py : médiane, gaussien, étirement, gamma, égalisation de la luminance.
- fourier.py : transformée 2D, filtres idéal et Butterworth, énergie haute fréquence locale.
- segmentation.py : masque HSV, masque de texture, score hybride.
- morphology.py : disque, érosion, dilatation, ouverture, fermeture, trous, petites régions.
- metrics.py : pourcentage, surface calibrée, IoU, précision, rappel, F1, exactitude.
- io_utils.py : lecture Pillow, masque de référence, image résultat fond noir et contour.
- demo.py : scène synthétique et son masque de vérité.
- report.py : tableau CSV et rapport PDF d'une analyse.

Fichiers d'interface

- app.py : fenêtre principale, navigation, fils d'exécution, dialogues de fichiers.
- ui/shell.py : barre latérale et en-tête, avec la barre de progression.

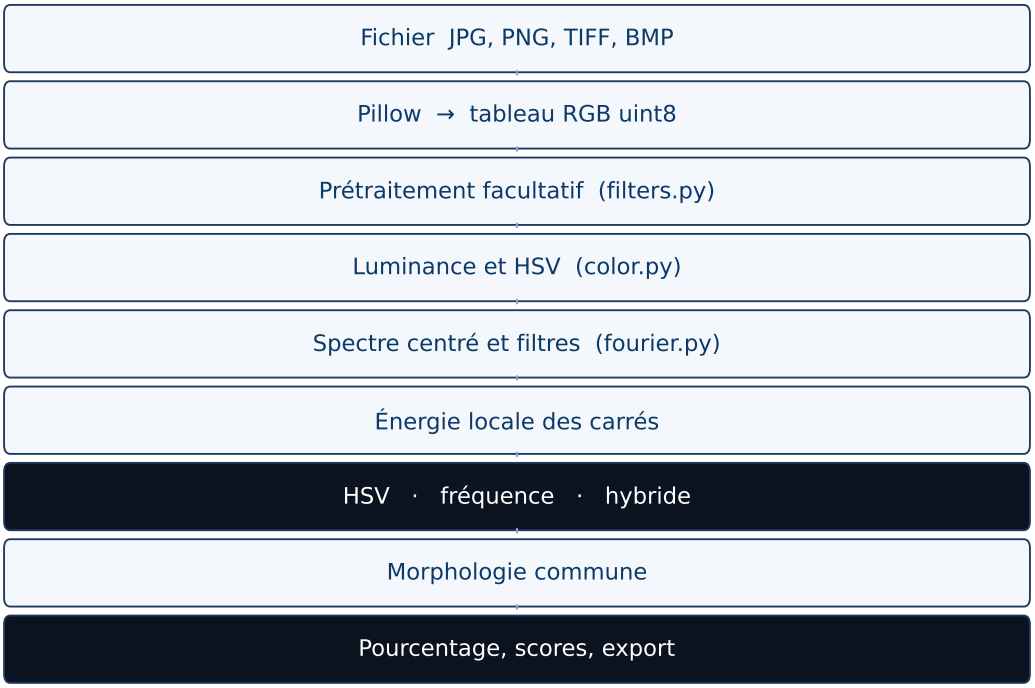
- ui/widgets.py : aperçus, curseurs, interrupteurs, cartes de chiffres, tableau.
- ui/charts.py : histogramme et diagrammes en barres sur fond sombre.
- ui/pages : un fichier par module, du tableau de bord jusqu'aux réglages.
- theme.py : couleurs bleu nuit et cyan, et les polices Segoe UI.

3. L'état d'une session

La classe AppState, dans state.py, est le cahier de la session. Elle conserve l'image originale, l'image de travail après prétraitement, le spectre complexe centré, la carte d'énergie locale, les masques bruts, les masques nettoyés, les métriques, le masque de référence et le dictionnaire de paramètres.
Un entier revision augmente chaque fois que l'image de travail change. Les calculs lancés en arrière-plan capturent cette valeur au départ. À leur retour, ils abandonnent le résultat si la révision a changé : l'utilisateur a chargé une autre photo pendant le calcul, et l'ancien masque ne doit pas recouvrir la nouvelle image.
set_original et set_working appellent _invalidate_products. Le spectre, les masques et les scores sont effacés, parce qu'ils décriraient une image qui n'est plus affichée.
Le masque de référence n'est effacé que lors d'un nouveau chargement, pas lors d'un simple prétraitement : la vérité terrain reste valable si les dimensions n'ont pas changé.
Le dictionnaire params regroupe tous les curseurs. DEFAULT_PARAMS donne l'identité du prétraitement : médiane de taille 1, gaussien nul, contraste « none ». L'image n'est donc pas modifiée tant que l'utilisateur n'appuie pas sur Apply. Les poids hybrides par défaut sont 0,50 pour la couleur, 0,35 pour la texture et 0,15 pour l'homogénéité. Le seuil de décision est 0,56. La somme texture plus homogénéité vaut 0,50, donc elle reste sous le seuil : une zone lisse sans la bonne couleur n'est pas classée comme eau.

4. Chaîne de traitement

Le schéma suivant est l'ordre réel des données. Le prétraitement est facultatif. La transformée de Fourier de l'image entière sert à visualiser et à filtrer. La carte locale sert à la segmentation. Les trois masques passent ensuite par la même morphologie, afin que la comparaison porte sur la décision et non sur le bruit résiduel.



5. Acquisition de l'image

io_utils.load_rgb ouvre le fichier avec Pillow. Les formats acceptés sont JPG, JPEG, PNG, TIFF et BMP. ImageOps.exif_transpose redresse les photographies de téléphone selon l'orientation EXIF, sinon le lac serait analysé couché. Un fichier TIFF de plusieurs pages ne fournit que la première, et le logiciel l'écrit dans le statut.
Une image avec transparence est composée sur du noir : un pixel transparent n'est pas une observation d'eau. Les autres modes sont convertis en RGB 8 bits. Pillow ramène aussi les TIFF 16 bits sur 0–255. Le tableau obtenu est un ndarray uint8 de forme (hauteur, largeur, 3), rendu contigu pour les calculs suivants. Une image plus petite que 8×8 pixels est refusée.
Dans l'interface, AcquisitionPage affiche ce tableau dans un canevas Tk. Le zoom est un seul facteur appliqué aux deux axes : à 100 %, l'image tient dans la zone ; au-delà, des barres de défilement apparaissent et le glisser déplace la vue. Le bouton Fit ramène le facteur à 1. Le panneau de droite indique la largeur, la hauteur, les trois canaux 8 bits et la taille du fichier.

6. Couleur : luminance et HSV

Luminance

La transformée de Fourier d'un manuel est définie sur un champ scalaire. color.rgb_to_gray réduit donc les trois canaux avant tout spectre. Les poids sont ceux de la recommandation UIT-R BT.601, classiques dans les cours de traitement d'images. Ils sont positifs et leur somme vaut 1, donc une image déjà dans [0, 1] y reste.

$$Y = 0{,}299\ R + 0{,}587\ G + 0{,}114\ B$$

Le vert domine parce que l'œil y est plus sensible. Un gris arithmétique (R+G+B)/3 sous-estimerait cette sensibilité. La fonction as_float01 divise un uint8 par 255 et laisse un tableau déjà dans [0, 1] tel quel, pour qu'une formule ne change pas de sens entre les deux échelles.

HSV

rgb_to_hsv applique la définition du cône hexagonal. Sur des canaux R, G, B ramenés à [0, 1], on calcule le maximum Cmax, le minimum Cmin et l'écart delta. La valeur V est Cmax. La saturation S est nulle si Cmax est nul, sinon delta/Cmax. La teinte H, en degrés, dépend du canal qui porte le maximum.

$$H = 60 \times (((G - B) / delta) \bmod 6) \quad \text{si le maximum est R}$$

$$H = 60 \times (((B - R) / delta) + 2) \quad \text{si le maximum est G}$$

$$H = 60 \times (((R - G) / delta) + 4) \quad \text{si le maximum est B}$$

Si delta vaut 0, le pixel est gris et la teinte est indéterminée. Le code range 0. Ce choix est sans effet, car la saturation est alors nulle et les tests de teinte ne doivent pas décider seuls. Le modulo ramène le secteur rouge, qui peut être négatif près du magenta, dans l'intervalle [0, 360).
HSV est utilisé parce qu'une même eau, plus profonde ou plus éclairée, garde à peu près la même teinte alors que ses valeurs RGB changent toutes les trois. L'intervalle par défaut isole le bleu : un pixel vert ou jaune n'y entre pas.

7. Prétraitement

filters.preprocess part toujours de l'image originale, pas de l'image déjà filtrée. Les curseurs décrivent donc un réglage absolu. L'aperçu est calculé dans un fil, et l'image de travail n'est remplacée que lorsque l'utilisateur appuie sur Apply to workspace. Les modules Fourier et Segmentation lisent cette image engagée.
L'ordre est fixe, et il est justifié par le type de défaut que chaque filtre sait enlever.

- La médiane passe en premier. Dans une fenêtre, elle remplace le pixel par la valeur centrale de la liste triée. Une impulsion isolée ne peut pas tirer cette valeur, contrairement à une moyenne. Le bruit poivre et sel est donc retiré avant qu'un filtre linéaire ne l'étale en halo. La taille du noyau est impaire. La valeur 1 désactive le filtre. SciPy calcule la médiane avec size=(k, k, 1) : les trois couleurs ne sont pas mélangées. Le bord est réfléchi, pour ne pas inventer un cadre sombre qui ressemblerait à une rive.
- Le gaussien vient ensuite. Son noyau continu est $G(x, y) = 1/(2\,\pi\,\sigma^2)\ \exp(-(x^2+y^2)/(2\,\sigma^2))$. SciPy l'échantillonne, le normalise, et l'applique avec un écart-type nul sur l'axe des couleurs. Une image constante est inchangée. Une sinusoïde de fréquence f est multipliée par $\exp(-2\,\pi^2\,\sigma^2\,f^2)$: plus f est grand,

plus l'atténuation est forte. σ est en pixels. En dessous de 0,05, l'étape est sautée.

- Le contraste est en dernier, pour ne pas amplifier le bruit que les deux filtres viennent de réduire. Quatre modes existent.

Le mode stretch étire chaque canal entre deux percentiles, par défaut 1 et 99. Quelques pixels saturés ne fixent donc pas l'échelle. C'est une application affine : elle ne crée pas de structure, elle consacre la dynamique d'affichage aux intensités réellement présentes.

$$C' = \mathrm{clip}\left((C - p_{bas}) / (p_{haut} - p_{bas}) , 0 , 1 \right)$$

Le mode gamma applique C' = C^γ sur [0, 1]. γ = 1 ne change rien. γ < 1 relève les ombres, car la dérivée est grande près de zéro. Les extrémités 0 et 1 restent fixes, et le même γ est utilisé sur les trois canaux pour préserver la teinte.
Le mode equalize calcule l'histogramme de la luminance sur 256 niveaux, puis sa fonction de répartition. La luminance égalisée est cdf(Y). Appliquer cette courbe séparément à R, G et B déplacerait la teinte. Le code multiplie plutôt le triplet RGB par le rapport Y'/Y, avec un rapport de 1 là où Y est nul. La direction chromatique reste, seule la clarté bouge.

8. Transformée de Fourier de l'image

fourier.fft2_shifted calcule la transformée discrète bidimensionnelle de la luminance, puis fftshift place la fréquence nulle au centre. Pour une image de M lignes et N colonnes, NumPy évalue

$$F(u, v) = \Sigma_x \Sigma_y \; f(x, y) \exp\left(-j \, 2\pi \, (u \, x / M + v \, y / N) \right)$$

La transformée inverse, ifft2, contient le facteur 1/(M N). Après fftshift, l'indice (M//2, N//2) est le terme continu F(0, 0), égal à la somme des pixels. La distance au centre est la fréquence spatiale : le milieu décrit les variations lentes, les coins décrivent la plus fine oscillation que la grille de pixels peut porter.
Le spectre est stocké en complex64. Le garder en mémoire permet de changer le filtre sans recalculer la transformée. reconstruct annule le décalage, applique l'inverse, et ne conserve que la partie réelle. La partie imaginaire est un résidu numérique, car un filtre réel et une image réelle produisent un spectre à symétrie hermitienne.

Visualisation

L'affichage utilise log(1 + |F|). La valeur absolue est la racine de la somme des carrés des parties réelle et imaginaire. Le terme continu est énorme, puisqu'il additionne tous les pixels. Le logarithme comprime cette dynamique, et le 1 intérieur évite le logarithme de zéro. Un étirement entre les percentiles 1 et 99,8 sert uniquement à l'œil : le filtre, lui, multiplie les coefficients complexes, jamais l'image logarithmique.

Filtres

La distance de chaque case au centre est D(u, v) = sqrt((u−uc)² + (v−vc)²). Le curseur de coupure est une fraction de la distance jusqu'au coin du spectre. Une fraction de 1 couvre donc toutes les fréquences, quelle que soit la taille de l'image.
Le passe-bas de Butterworth d'ordre n vaut 1 à l'origine, donc il conserve la moyenne. Il vaut 1/2 lorsque D = D0, pour tout ordre. Un ordre élevé rapproche la transition d'un disque idéal.

$$H_{pb}(u, v) = 1 / (1 + (D(u, v) / D0) ^ (2 n))$$

Le passe-haut classique n'est pas exactement 1 − H_pb, mais il a le même rayon à mi-puissance. Au centre, D0/D n'est pas défini : le code écrit 0 explicitement, ce qui retire la moyenne.

$$H_{ph}(u, v) = 1 / (1 + (D0 / D) ^ (2 n)) \quad \text{pour } D > 0$$

Le filtre idéal est un disque dur : 1 à l'intérieur du rayon, 0 dehors. Sa transformée inverse est un jinc, et la convolution avec ce noyau produit des ondulations de Gibbs le long des contours francs. Il est proposé parce que c'est la définition vue en premier dans un cours. Le Butterworth est le réglage par défaut.
Multiplier F par H réel ne change pas la phase, donc le filtre atténue des fréquences sans décaler les contours. Un passe-bas reconstruit une luminance lissée, affichée sur l'échelle d'origine. Un passe-haut reconstruit un résidu de moyenne nulle, avec des valeurs négatives. Pour le montrer, le résidu est étiré sur [0, 1], et le cartouche de l'image le dit : cet étirement n'est pas l'image filtrée elle-même.

9. Énergie haute fréquence locale

La question n'est plus « comment filtrer toute la photo », mais « dans ce petit voisinage, quelle part de l'énergie est rapide ». local_high_frequency_ratio découpe la luminance en carrés. Le pas est choisi pour qu'il y ait au plus max_frequency_windows fenêtres le long de chaque axe, par défaut 56. Une grande photo ne demande donc pas une transformée par pixel.
Chaque carré est centré, c'est-à-dire privé de sa moyenne, puis multiplié par une fenêtre de Hann séparable. La fenêtre w(n) = 0,5 − 0,5 cos(2π n / (N−1)) vaut zéro aux bords. Sans elle, la transformée traiterait le carré comme une période d'un signal périodique, et le saut aux bords fuirait dans les hautes fréquences. C'est la fuite spectrale. La même fenêtre est appliquée partout, donc les rapports restent comparables.
On calcule ensuite le spectre du carré et sa densité |P|². Le théorème de Parseval dit que la somme de |P|² est proportionnelle à l'énergie du carré fenêtré. Le rapport de deux sommes est donc un rapport d'énergie. Un disque de rayon cutoff_ratio × (côté / 2) sépare les basses fréquences du reste.

$$E_{haut} / E_{totale} = \Sigma_{D > D0} |P|^2 \quad / \quad \Sigma |P|^2$$

Un carré parfaitement plat a une énergie totale presque nulle après retrait de la moyenne. Son rapport est défini comme 0. Un feuillage ou un bruit a un rapport élevé.
La carte grossière est ensuite agrandie par interpolation bilinéaire jusqu'à la taille de l'image. Cet agrandissement ne crée pas de nouvelles mesures : il permet seulement aux règles pixel à pixel de lire une valeur.
Cette carte décrit une texture. Elle ne détecte pas l'eau. Le sable, un toit, une route et une ombre plate produisent aussi un rapport faible. L'interface l'écrit sur la page Fourier et sur l'onglet Frequency de la segmentation.

10. Trois segmentations

segmentation.segment calcule les trois masques sur la même image de travail, puis leur applique la même morphologie. Les masques bruts et les masques nettoyés sont tous conservés.

Méthode 1 — seuils HSV

hsv_hard_mask est un test booléen. Un pixel est retenu s'il est dans la boîte primaire, ou dans la seconde boîte si elle est activée, ou dans la règle d'eau sombre si elle est activée. La boîte primaire par défaut couvre les teintes de 185° à 255°, donc le bleu, avec une saturation d'au moins 0,12 et une valeur entre 0,08 et 0,92. Le vert et le jaune tombent en dehors. La seconde bande et la règle d'eau sombre sont éteintes tant que l'utilisateur ne les active pas. Le plafond de valeur écarte les nuages et la mousse très clairs.
La teinte est un angle. hue_in_range sait traverser 0° : si le minimum est supérieur au maximum, l'arc passe par le rouge, et un pixel est dedans s'il est au-dessus du minimum ou en dessous du maximum. La seconde bande, éteinte par défaut, est un ocre étroit, de 8° à 42°, avec une valeur maximale de 0,55. Elle vise une eau chargée de sédiments, pas un sable clair dont la valeur dépasse ce plafond.
La règle d'eau sombre accepte un pixel seulement s'il est à la fois peu lumineux et peu saturé. La teinte d'un gris est instable. Un végétal sombre mais saturé échoue au test de saturation. Un trottoir gris clair échoue au test de valeur.

Méthode 2 — candidat fréquentiel

Le masque vaut vrai là où le rapport d'énergie haute fréquence est strictement inférieur au seuil de texture, 0,42 par défaut. Le nom du masque dans l'interface est « Fourier candidate ». C'est volontaire : le résultat montre les régions lisses. Il ne constitue pas, à lui seul, une détection d'eau.

Méthode 3 — score hybride

C'est la méthode prévue pour la surface annoncée. Chaque pixel reçoit un score entre 0 et 1, combinaison convexe de trois termes. Les poids affichés sont renormalisés pour

sommer à 1. S'ils sont tous nuls, le code retombe sur la couleur seule, afin que le score reste défini.

$$S = w_c\, C + w_t\, T + w_h\, H \quad \text{avec } w_c + w_t + w_h = 1$$

C est une appartenance douce aux mêmes bandes HSV. Hors d'un intervalle, la teinte décroît comme une gaussienne dont l'écart est hue_softness, 8° par défaut. La saturation et la valeur décroissent de la même façon, avec un écart de 0,06. Les trois portes se multiplient : C = C_H × C_S × C_V. Un sable clair ou un continent vert est loin de l'arc bleu, donc le produit tombe et le terme de texture ne suffit plus à franchir le seuil. La règle sombre, si on l'active, est ajoutée par un maximum, pas par ce produit, parce qu'un gris n'a pas de teinte exploitable.
T est la version souple du seuil de la méthode 2. C'est une logistique centrée sur ce seuil. En dessous, le pixel est lisse et T approche 1. Au-dessus, T approche 0. La pente texture_steepness vaut 14 par défaut. L'argument de l'exponentielle est borné pour éviter un dépassement numérique.

$$T(E) = 1 / (1 + \exp(k\,(E - \text{seuil})))$$

H mesure l'homogénéité spatiale, qui n'est pas le même énoncé que le rapport de Fourier. La variance locale dans une boîte est $E[X^2] - (E[X])^2$, les deux espérances étant des filtres uniformes SciPy. L'écart-type est comparé à une référence, 0,08 sur l'échelle [0, 1].

$$H = \text{clip}(1 - \sigma / \sigma_{ref} , 0 , 1)$$

Un dégradé lent au bord d'un lac a de la variance mais presque pas de haute fréquence. Des rides ou du feuillage ont les deux. Le terme H est facultatif. Le pixel est eau lorsque S est au moins égal au seuil de décision. Avec les poids par défaut, la couleur doit participer : 0,35 + 0,15 = 0,50, et 0,50 est inférieur à 0,56.

11. Morphologie

morphology.py construit le disque avec NumPy : un pixel à l'offset (y, x) appartient à l'élément structurant lorsque $y^2 + x^2 \le r^2$. Le rayon 0 donne un seul pixel, et toutes les opérations deviennent l'identité. SciPy réalise l'érosion et la dilatation binaires. L'ouverture et la fermeture sont composées dans ce fichier, pour que l'ordre reste visible.

$$\text{Érosion : } A \ominus B = \{\, z \mid B_z \text{ est inclus dans } A \,\}$$

$$\text{Dilatation : } A \oplus B = \{\, z \mid A \text{ rencontre } B_z \,\}$$

$$\text{Ouverture : } (A \ominus B) \oplus B \qquad \text{Fermeture : } (A \oplus B) \ominus B$$

L'extérieur de l'image est traité comme du fond. Les objets rétrécissent, et un objet qui touche le bord n'est pas artificiellement conservé. L'ouverture passe avant la

fermeture. Elle efface les faux positifs plus fins que le disque avant que la fermeture puisse les souder à un lac. La fermeture bouche ensuite les cassures dues aux reflets. Le rayon par défaut est 2.
Le remplissage des trous utilise binary_fill_holes. Le complément est inondé depuis le bord. Tout pixel de fond que l'inondation n'atteint pas est entouré d'eau et rejoint le masque. Une baie qui touche le bord de la photo n'est pas un trou : une embouchure ou un lac coupé par le cadre reste ouvert.
Les composantes sont étiquetées en connexité 8, avec un élément 3×3 de uns. Un fil d'eau en diagonale forme donc une seule région, ce qui correspond à une rivière qui traverse la grille. Les composantes dont le nombre de pixels est strictement inférieur à min_region_pixels, 80 par défaut, sont supprimées en dernier. Ce seuil est un nettoyage en pixels. Ce n'est pas une surface au sol.

12. Surface, calibration et scores

Compte des pixels

Tous les pixels de l'image de travail sont valides. Le logiciel n'invente pas de masque de données manquantes. Un cadre noir ou un cartouche fait partie du dénominateur.

$$\text{pourcentage} = (\text{pixels d'eau} / \text{pixels de l'image}) \times 100$$

La surface physique est un autre énoncé. Elle exige une distance au sol : la largeur et la hauteur, en mètres, représentées par un pixel.

$$\text{surface_m}^2 = \text{pixels d'eau} \times \text{largeur_pixel_m} \times \text{hauteur_pixel_m}$$

Si la calibration est éteinte, ou si une des deux longueurs n'est pas finie et strictement positive, la surface en mètres carrés reste indéfinie. L'interface affiche alors une phrase explicite : le résultat est un pourcentage de pixels, pas une aire au sol. Lorsque la calibration est active, la note rappelle que l'échelle est supposée constante et qu'aucune pente de terrain n'est corrigée. L'hectare affiché est simplement le nombre de mètres carrés divisé par 10 000.
Le nombre de régions et la taille de la plus grande viennent de l'étiquetage 8-connexe du masque nettoyé. La méthode active, hybride par défaut, alimente les cartes du tableau de bord. Le diagramme en barres compare les trois pourcentages.

Vérité terrain

Un masque de référence est une image dont les pixels au-dessus de 127, ou au-dessus de 0,5 s'il est flottant, sont l'eau. S'il n'a pas la taille de l'image de travail, resize_nearest le rééchantillonne au plus proche voisin. Une interpolation linéaire inventerait des gris sans signification pour une étiquette. L'interface et le rapport disent que ce redimensionnement a eu lieu.
Pour une prédiction P et une référence G, les quatre effectifs sont les cardinaux des

intersections.

$$TP = |P \cap G| \quad FP = |P \cap \text{complément de } G|$$

$$FN = |\text{complément de } P \cap G| \quad TN = |\text{complément de } P \cap \text{complément de } G|$$

$$\text{précision} = TP / (TP + FP) \quad \text{rappel} = TP / (TP + FN)$$

$$F1 = 2\,PR / (P + R) \quad IoU = TP / (TP + FP + FN)$$

$$\text{exactitude} = (TP + TN) / N$$

Le F1 d'une seule classe binaire est le coefficient de Dice. Si un dénominateur est nul parce que les deux masques sont vides, le score vaut 1 : il n'y avait pas d'eau et aucune eau n'a été prédite. L'exactitude est affichée parce qu'elle est demandée, avec un avertissement : quand l'eau occupe peu de la photo, les vrais négatifs la maintiennent haute même si le rivage est mauvais. L'IoU et le F1 décrivent la région d'eau.
L'image d'accord colorie ces quatre ensembles : cyan pour les vrais positifs, ambre pour les faux positifs, violet pour les faux négatifs, et le fond sombre pour les vrais négatifs.

13. Image résultat : fond noir et contour

overlay_mask, dans io_utils.py, construit l'image montrée à droite du tableau de bord et enregistrée comme image annotée. Tout pixel hors du masque devient noir. Les pixels d'eau gardent leur couleur d'origine, donc la surface reste visible. Le contour est le gradient morphologique, une bande de deux pixels située à l'intérieur de l'eau.

$$\text{contour} = A \text{ privé de } (A \text{ érodé deux fois})$$

Cette bande est peinte en blanc, sur l'eau. Le paramètre alpha de la fonction n'est plus utilisé : le résultat est une séparation franche, pas un voile translucide posé sur toute la photographie.

14. Interface

Fenêtre

WaterVisionApp, dans app.py, est une fenêtre CustomTkinter. Le thème est forcé en sombre avant la création des widgets. La grille a deux colonnes : la barre latérale, large d'environ 230 pixels, et la zone principale. L'en-tête porte le nom du module, une phrase d'explication, le nom du fichier et le statut. Une barre de progression déterminée reste au repos ; elle passe en mode indéterminé seulement pendant un calcul.

Les neuf pages sont créées une fois et empilées dans la même cellule. navigate affiche la page demandée et retire les autres avec grid_remove. Elle resynchronise les curseurs depuis params, puis rafraîchit les aperçus. Revenir sur Fourier ne relance la transformée que si la révision de l'image a changé.

Pages

- Dashboard. Cinq cartes : pourcentage, pixels d'eau, nombre de régions, plus grande région, durée de la segmentation. À gauche, l'image originale. À droite, l'eau sur fond noir. Le bandeau sous les cartes est ambre tant qu'aucune taille de pixel n'est fournie, et vert lorsqu'une surface en mètres carrés existe.
- Image Acquisition. Chargement, scène de démonstration, zoom, ajustement à la vue, remise à zéro de tout l'espace de travail.
- Preprocessing. Deux aperçus et l'histogramme de la luminance de l'aperçu. Les curseurs mettent à jour un calcul différé de 280 ms. Le texte d'avertissement rappelle que l'aperçu n'est pas encore l'image de travail.
- Fourier Analysis. Quatre images : photo de travail, spectre logarithmique, image filtrée, carte d'énergie locale. Les curseurs de filtre relancent seulement le produit H×F et la transformée inverse, grâce au spectre mis en cache.
- Segmentation. Grille 2×2 : image, masque HSV, candidat fréquentiel, masque hybride. Les réglages sont dans quatre onglets. Le sélecteur HSV / Fourier / Hybrid choisit le masque utilisé pour la surface, le tableau de bord et l'export par défaut. Run segmentation lance les trois méthodes. Reset parameters restaure les réglages de segmentation et retire les masques devenus incohérents.
- Area Estimation. Les cartes résument la méthode active. Le tableau et le diagramme comparent les trois. Apply pixel size valide deux nombres strictement positifs, ou éteint la calibration.
- Evaluation. Charge un masque de référence, montre l'accord coloré, le tableau des scores et le diagramme groupé IoU, précision, rappel et F1.
- Export. Masque PNG 0/255, image annotée, spectre, CSV et PDF d'analyse. Le CSV et le PDF acceptent une image sans segmentation : les sections absentes sont marquées comme non calculées, elles ne sont pas inventées.
- Settings. Nombre maximal de fenêtres fréquentielles, palette du spectre, texte de rappel sur ce que chaque mesure affirme, et restauration de tous les paramètres.

Contrôles

SliderRow affiche le nom, la valeur formatée et le curseur. Pendant une remise à jour programmée, un drapeau silencieux empêche le curseur de relancer le calcul qui est en train de l'initialiser. SwitchRow fait de même pour les interrupteurs. Les info-bulles sont de petites fenêtres Tk sans bordure : CustomTkinter n'en fournit pas. ImageCard redimensionne l'aperçu avec Pillow en conservant le rapport largeur/hauteur, puis le donne à un CTkImage de cette taille exacte. Le widget n'étire donc pas la photo.
Un défaut de Tkinter apparaît si l'on détruit le CTkImage pendant que l'étiquette l'affiche encore : le message « image pyimageN doesn't exist ». ImageCard conserve

l'objet précédent jusqu'à ce que l'étiquette ait reçu la nouvelle image. Une pastille de 1×1 pixel, jamais détruite, remplace l'aperçu lorsqu'il n'y a rien à montrer.

15. Calculs et interface : qui appelle qui

Une page ne lance pas elle-même un fil. Elle appelle une méthode de WaterVisionApp : open_image, load_demo, compute_spectrum, compute_local_energy, run_segmentation, set_calibration, ou un export. run_task refuse un second gros calcul tant que le drapeau busy est vrai, et une boîte de dialogue le dit. Le travail s'exécute dans un fil démon. Le résultat revient sur le fil de Tk par after(0, ...). Seul ce fil touche aux widgets.

Le filtre de Fourier est une exception légère. Changer la coupure ne prend pas le drapeau busy. Un compteur de génération jette les résultats périmés si l'utilisateur a encore bougé le curseur. Le spectre complexe, lui, n'est pas recalculé.

open_image ouvre le sélecteur avec la fenêtre WaterVision comme parent, et les motifs de fichiers sont séparés par des points-virgules, forme attendue par le dialogue Windows. Après un chargement réussi, l'application affiche tout de suite le tableau de bord, puis lance la segmentation de cette photo. Le nom de fichier dans l'en-tête devient celui du fichier choisi. La scène de démonstration suit le même retour au tableau de bord, pour montrer le lac sur fond noir plutôt que de laisser l'utilisateur sur une autre page.

Les graphiques sont des objets matplotlib.figure.Figure, pas des figures pyplot. Le canevas TkAgg est dessiné sur le fil principal. Le PDF d'analyse, lui, est construit hors écran dans le fil de l'export, avec le backend PDF. Les deux ne partagent pas l'état global de pyplot.

16. Scène de démonstration

demo.make_demo_scene fabrique une image 480×640 et son masque de vérité, sans fichier extérieur. La graine aléatoire est fixe. Le sol est un jaune-vert texturé, volontairement hors de la bande bleue. Un banc de sable est lisse et beige : la méthode fréquentielle l'accepte, la teinte et le score hybride le refusent. Le lac et la rivière sont bleus, avec un très faible bruit, et la teinte reste près de 200°.

L'étang gris n'est pas bleu : avec la règle d'eau sombre éteinte, il n'est pas retenu.

La vérité terrain est l'union géométrique du lac, de la rivière et de l'étang. Le sable en est exclu. Le test test_methods_separate_smooth_sand_from_water vérifie que l'hybride couvre le lac, que Fourier couvre le sable, et que l'hybride ne traite pas ce sable comme de l'eau. Il vérifie aussi que l'IoU hybride dépasse l'IoU fréquentiel sur cette scène.

17. Exports d'une analyse

write_csv écrit trois blocs dans un fichier UTF-8 avec indicateur d'ordre des octets, pour qu'Excel sous Windows reconnaisse l'encodage : le tableau des surfaces, les

scores s'ils existent, puis tous les paramètres. Une surface sans calibration est marquée « pixel-based estimate » et la colonne en mètres carrés reste vide. write_pdf assemble un rapport d'analyse, distinct du présent document. Les pages contiennent le rappel de la formule du pourcentage, l'image de travail, l'eau sur fond noir, les trois masques, le spectre s'il a été calculé, les scores s'il existe une référence, puis la liste des paramètres et le rôle de chaque méthode. Une section absente est une phrase, pas une figure inventée.

18. Lancer, vérifier, limites

Installation et lancement

```
python -m pip install -r requirements.txt
python main.py
python -m unittest discover -s tests -t .
```

Les dépendances sont NumPy, SciPy, Pillow, Matplotlib et CustomTkinter. Python 3.11 ou plus récent est requis. Les tests du dossier tests ne créent pas de fenêtre : ils contrôlent les teintes primaires, l'inversion de la transformée de Fourier, l'ouverture morphologique, le refus d'une calibration nulle, et le comportement de la scène synthétique.

Ce que le logiciel ne prétend pas faire

- Il ne géoréférence pas une image. Sans largeur et hauteur de pixel, il n'y a pas de mètres carrés.
- Il ne corrige pas le relief. Un pixel en pente ne représente pas la même aire au sol qu'un pixel plat.
- Il ne distingue pas, par la fréquence seule, l'eau d'une autre surface lisse.
- Une eau très ridée peut perdre le terme de texture. L'utilisateur peut alors baisser le seuil de décision ou augmenter le poids de la couleur. Le vert et le jaune restent hors de l'intervalle bleu. Pour les reprendre, il faut élargir la teinte, ou activer la seconde bande.
- Le prétraitement et la segmentation décrivent la photo telle qu'elle est cadrée. Un cadre noir compte dans le total des pixels.

Les formules détaillées, avec les cas limites numériques, restent dans les docstrings des modules de watervision/processing. Ce document est le plan qui permet de les lire dans l'ordre.